Licence Devops Année 2025 - 2026

Architecture N tiers

Les objectifs du module

- Étude des architectures applicatives / Bonnes pratiques agiles de développement
- Maven
- Serveurs d'applications : Tomcat
- Web Services (XML/SOAP, REST) : Rest
- Jira / Confluence

Les objectifs Architectures applicatives

- Pré requis : différentes étapes de développement, cycle de développement, …
- Architecture
- SOA, ESB
- Outils

Les différentes étapes de développement

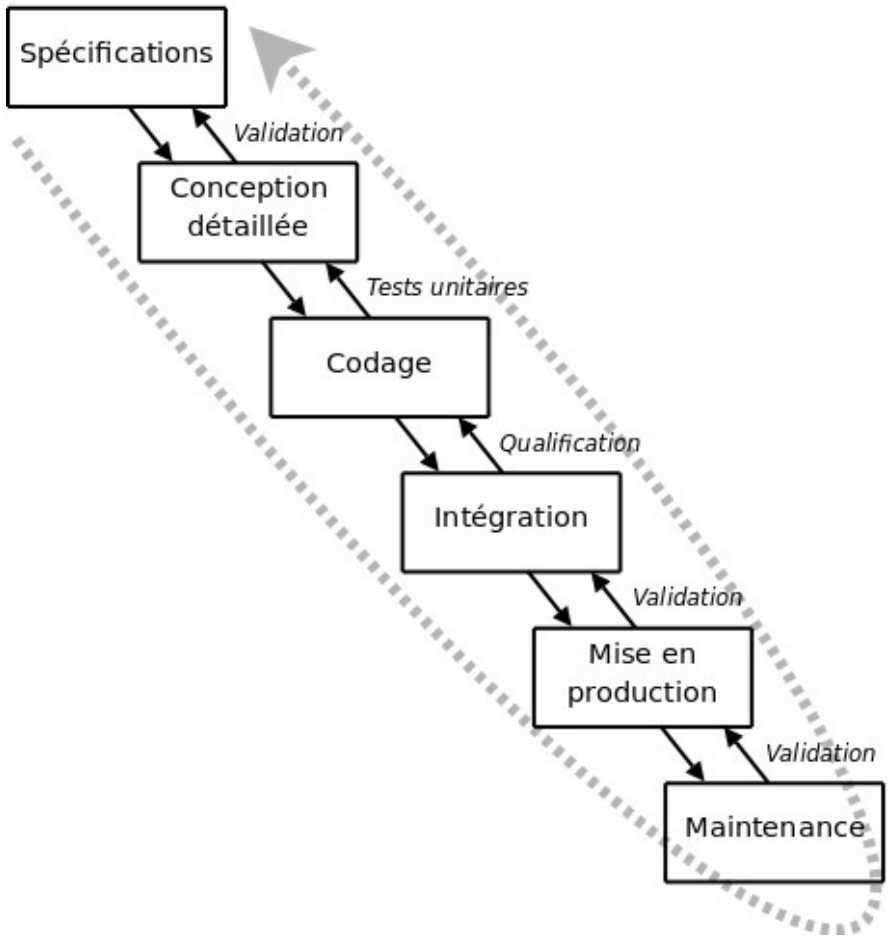
- Analyse des besoins
- Spécifications
- Conception de l’architecture
- Conception détaillée
- Implémentation
- Tests Unitaires
- Test d’intégration
- Test de validation (Métiers & Utilisateurs)
- Maintenance

Les Cycles de développement

- Cascade

- **Avantages du modèle en cascade :**
  - Le planning
  - La MOA connait le contenu de la livraison
- **Inconvénients du modèle en cascade :**
  - les anomalies sont détectées tardivement

    => Coût important en cas d'anomalie.

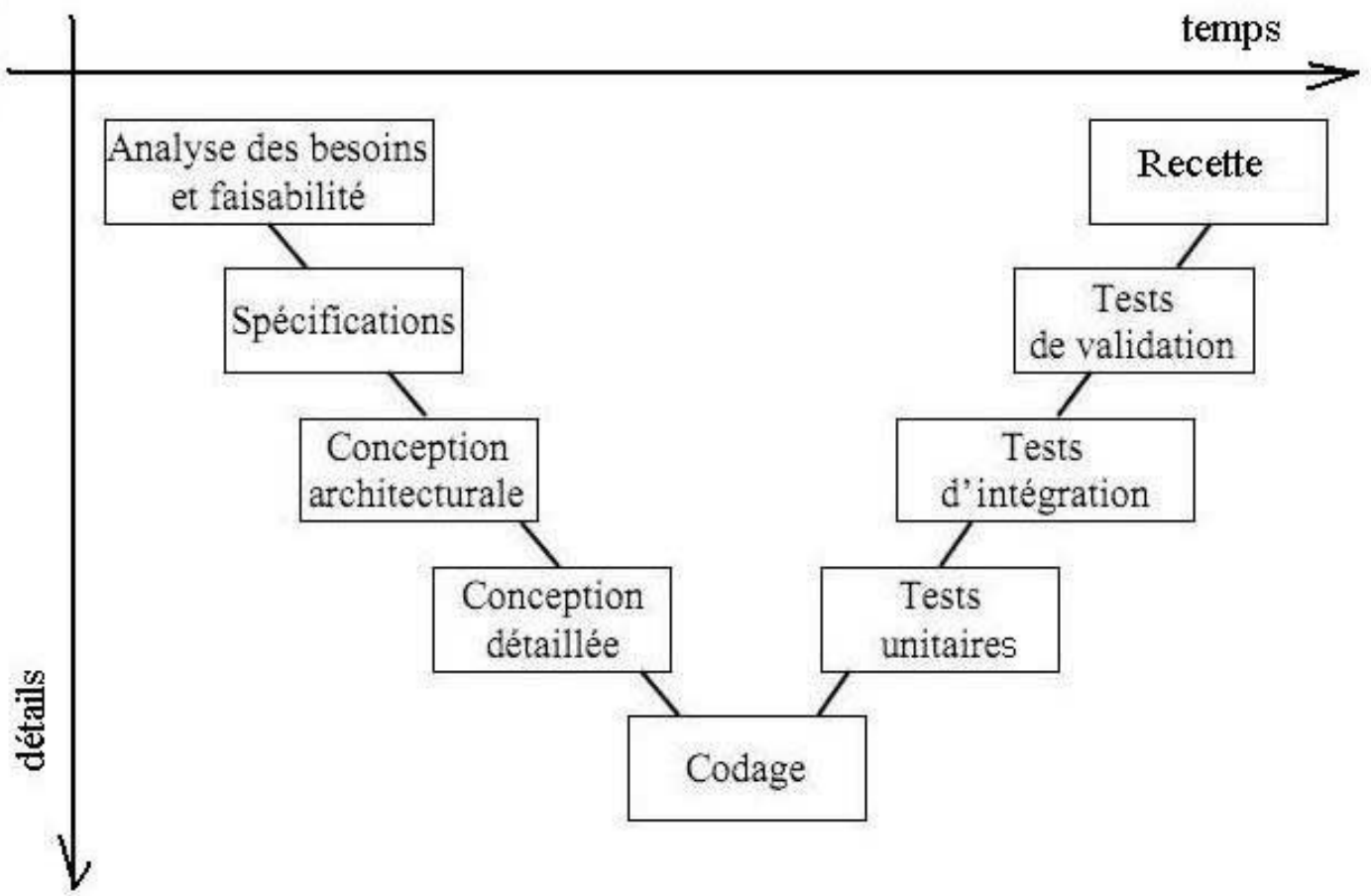


Les Cycles de développement

- Cycle en V

- **Avantage du modèle en V :**
  - le livrable final doit être parfait.
    => plusieurs étapes de réflexion + tests

- **Inconvénients du modèle en V :**
  - **Un déséquilibre**
  - **Le retour d'anomalie coute cher.**
  - **Le temps**

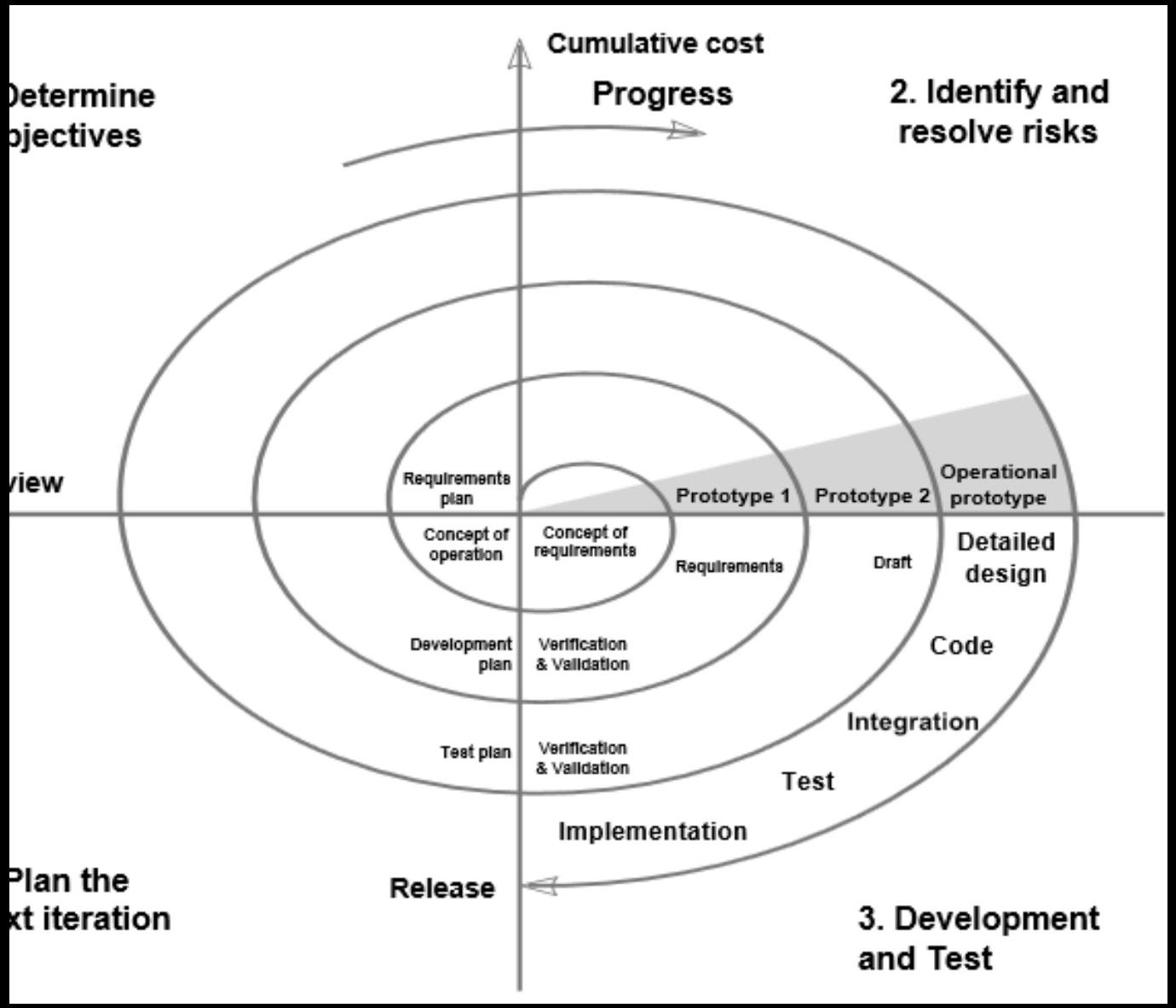


Les Cycles de développement

Cycle en Spirale

- Inconvénient :

La mise en place est difficile sur les petits projets.



Les Cycles de développement

- Agilité : les quatre valeurs fondamentales
  - L’équipe : Equipe soudée, communique
  - L’application : Vis-à-vis de la documentation
  - La collaboration : Implication du client (ses attentes)
  - L’acceptation du changement

Les Cycles de développement

Agilité : les principes

- La plus haute priorité est de satisfaire le client en livrant rapidement et régulièrement des fonctionnalités à forte valeur ajoutée.
- Le changement est accepté, même tardivement dans le développement.
- La livraison s’applique à une application fonctionnelle, toutes les deux semaines à deux mois, avec une préférence pour la période la plus courte.
- Le métier et les développeurs doivent collaborer régulièrement et de préférence quotidiennement au projet.
- Le projet doit impliquer des personnes motivées.
- La méthode la plus efficace pour transmettre l'information est une conversation en face à face.
- L’unité de mesure de la progression du projet est un logiciel fonctionnel
- Les processus agiles promeuvent un rythme de développement soutenable (afin d’éviter la non qualité découlant de la fatigue).
- Les processus agiles recommandent une attention continue à l'excellence technique et à la qualité de la conception.
- La simplicité et l'art de minimiser les tâches parasites, sont appliqués comme principes essentiels.
- Les équipes s'auto-organisent afin de faire émerger les meilleures architectures, spécifications et conceptions.
- À intervalle régulier, l'équipe réfléchit aux moyens de devenir plus efficace, puis accorde et ajuste son processus de travail en conséquence.

Les Cycles de développement

Système d’information && d’informatique

- Différences entre le Système d’information et le système d’informatique
  - **<u>Système informatique</u>** : Description des outils informatiques qui traiterons l'information. Il est composé du matériel informatique (machines et réseaux) et des logiciels installés sur ces matériels.
  - **<u>Système d’information</u>** : description de l'ensemble des informations et des flux d'information dans une entreprise. Cela comprend l’ensemble des outils mais également les actifs humains et immatériels, les procédés, procédures, et processus, l’organisation, …

Urbanisme

- L’urbanisation d’un système d’information est une démarche permettant de faire évoluer le système d’information.
- Son but :
  - Garantir les objectifs métiers
  - Garantir la cohérence des applications
- Avantages :
  - Maitriser le SI
  - Maitriser les coûts
  - Evoluer les SI en fonction des besoins Métiers.

Architecture logicielle

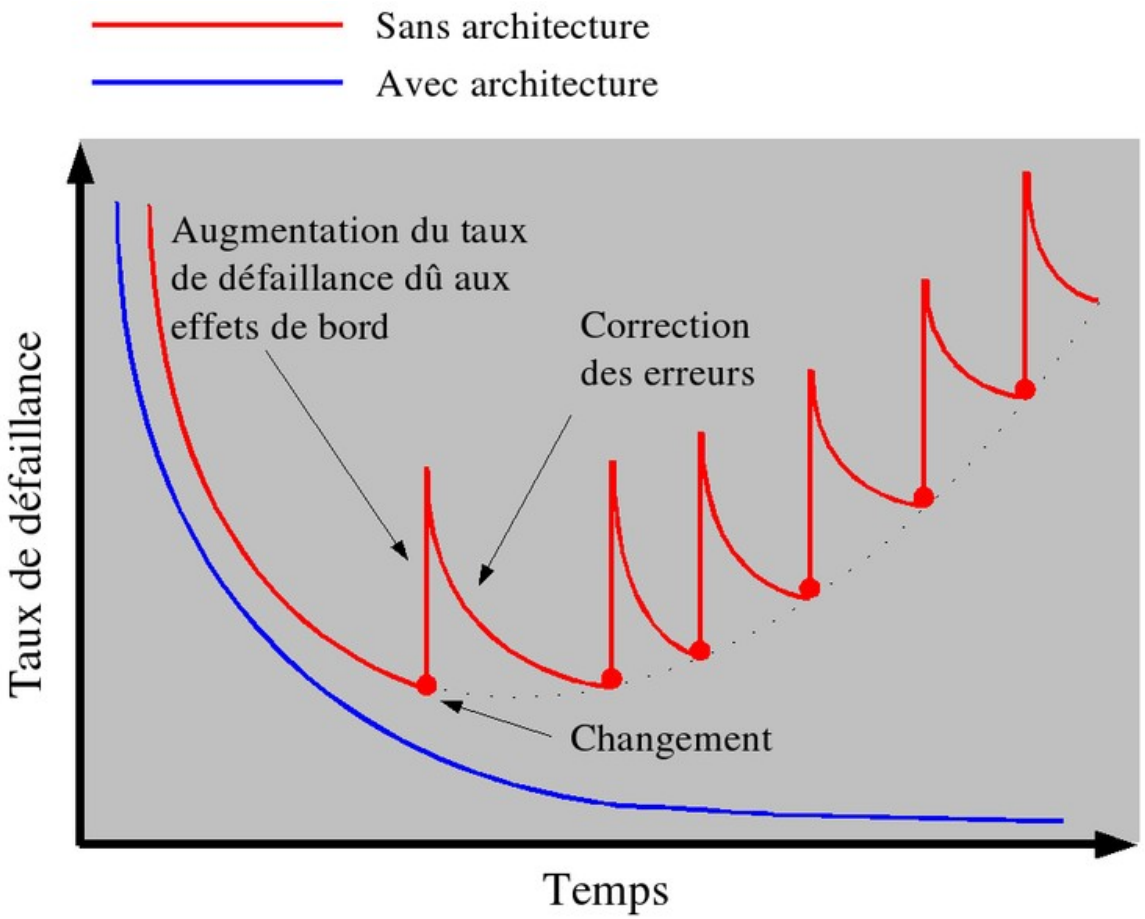
- L'art de construire les logiciels
- Détermine la stabilité, sa performance et sa pérennité.
- Différence
  - L’architecture fonctionnelle (analyse) décrit le « quoi faire ».
  - l’architecture logicielle décrit le « comment le faire ».
- et la conception …
  - l’architecture correspond à

Architecture logicielle / conception

S.No.	CONCEPTION DE LOGICIELS	ARCHITECTURE LOGICIELLE
01.	La conception de logiciels consiste à concevoir des modules/composants individuels.	L'architecture logicielle concerne l'architecture complète du système global.
02.	La conception du logiciel définit les propriétés détaillées.	L'architecture logicielle définit les propriétés fondamentales.
03.	En général, cela fait référence au processus de création d'une spécification d'artefact logiciel qui aidera les développeurs à implémenter le logiciel.	En général, il fait référence au processus de création de la structure de haut niveau d'un système logiciel.
04.	Il aide à mettre en œuvre le logiciel.	Il aide à définir l'infrastructure de haut niveau du logiciel.
05.	La conception du logiciel évite l'incertitude.	L'architecture logicielle gère l'incertitude.
06.	La conception de logiciels concerne davantage les modules/composants individuels.	L'architecture logicielle concerne davantage la conception de l'ensemble du système.
07.	En un mot, le niveau de conception du logiciel est la mise en œuvre.	En un mot, le niveau de l'architecture logicielle est la structure.

source : https://stacklima.com/

Architecture logicielle

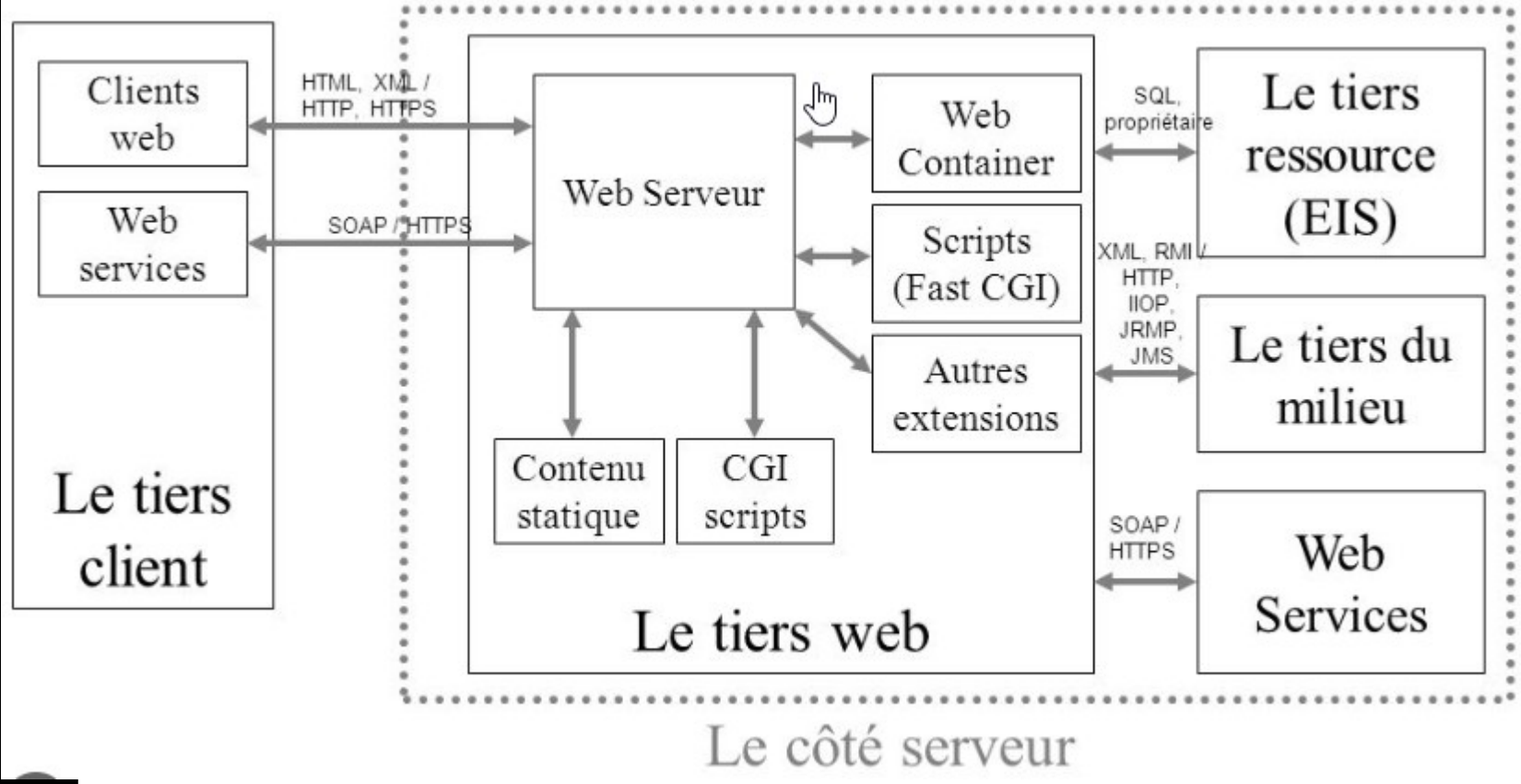


Architecture logicielle

Exercice :

- Schématiser une architecture logicielle

Architecture logicielle



Architecture N tiers

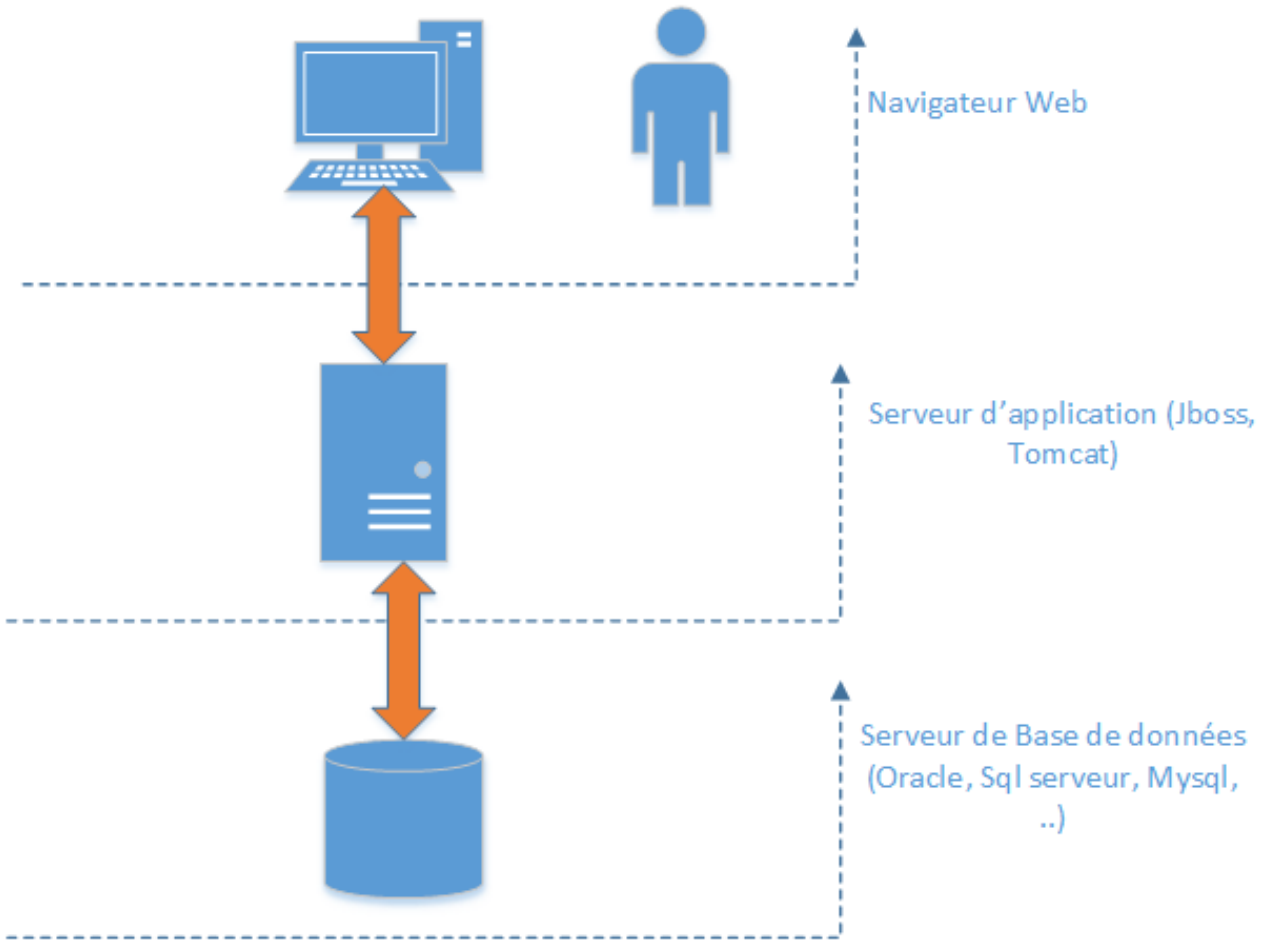
- **Architecture 2 tiers, Architecture 3 tiers , Architecture n tiers**
- **A partir de l'architecture 3 tiers :**

  - Flexibilité d'architecture matériel
  - meilleur performance
  - Extensibilité, Sécurité
  - Réutilisation de certains modules
  - Sources de données hétérogènes (plusieurs bd, …)
  - disponibilité

Séparer les responsabilités avec une architecture multi-tiers

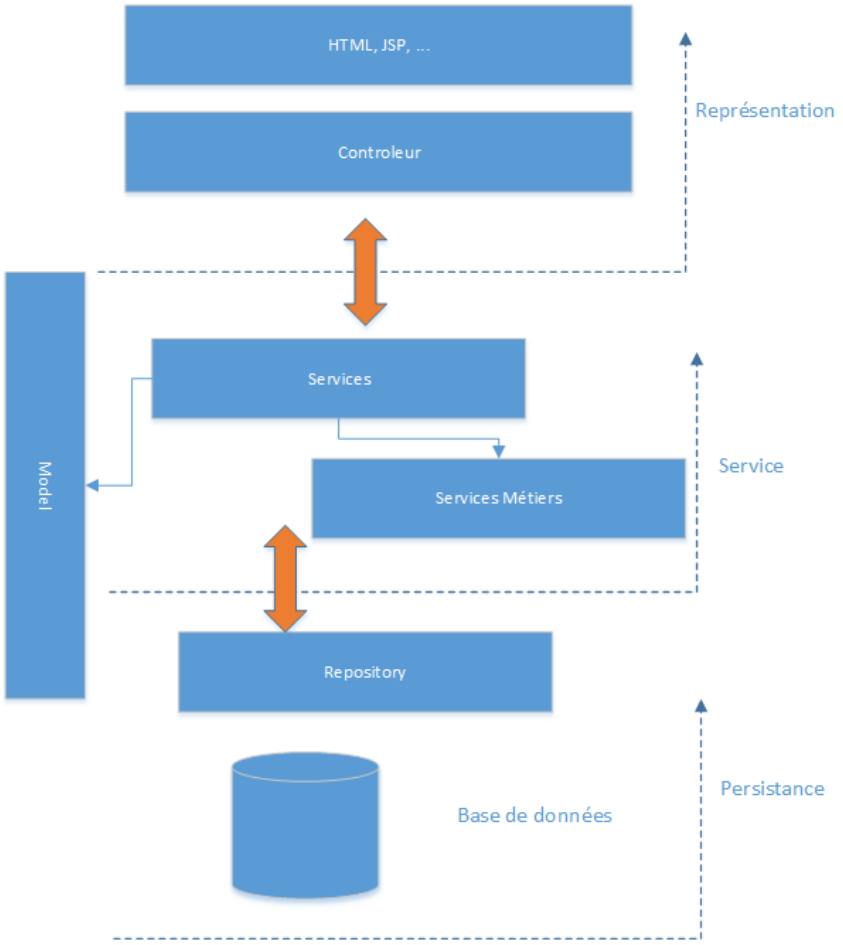
Architecture N tiers

- La couche physique : répartition sur plusieurs serveurs
- Le serveur d'application : couche Présentation + Métiers + couche d'accès à la base de données (persistance).

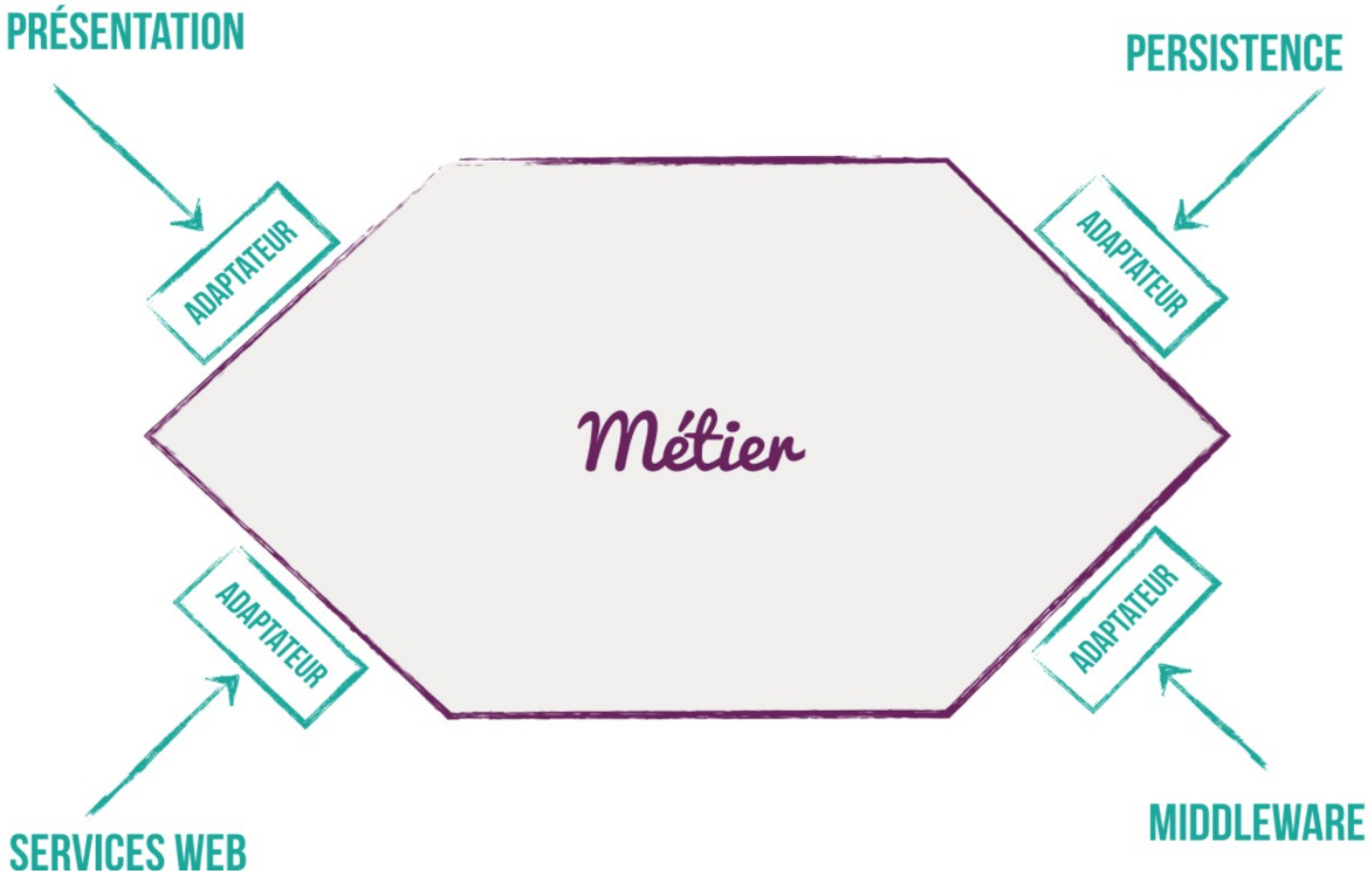


Couche logicielle

- La couche Présentation : s’occupe de l’implémentation « Interface » à destination d’un client riche ou d’un client Web.
- La couche Service: contient les règles métiers.
- La couche Persistance : gère l'accès aux sources de données



Architecture Hexagonale



Architecture Hexagonale

Bilan

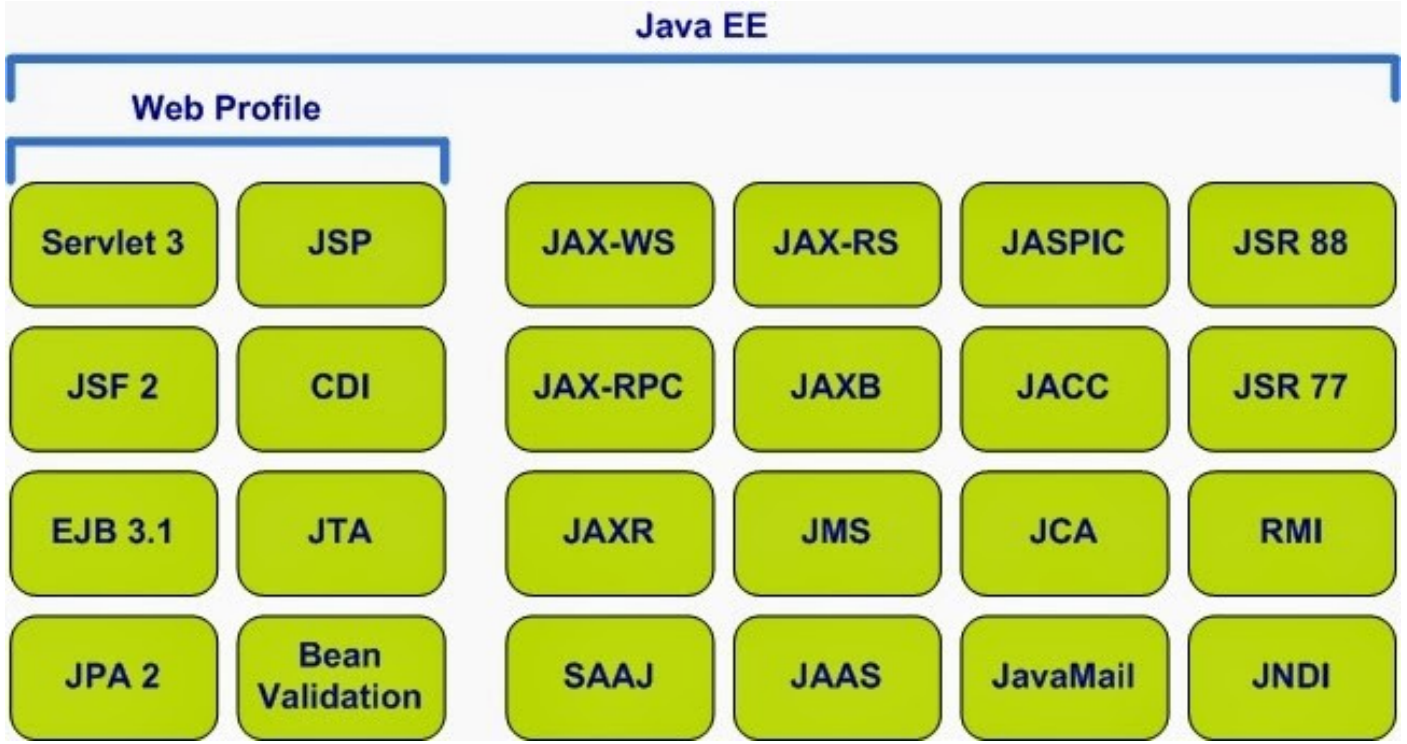
- Design Pattern : bonnes pratiques
  - Les développeurs peuvent communiquer entre eux.
  - Cela favorise de réutiliser des modules
  - Les différentes couches facilite les modifications de conception.
- Phase de conception et d’architecture : primordial
  - Réduire la complexité pour éviter une boite noire.
    - Facilite la maintenance
    - Réduit le temps de correction d’anomalies.
  - Analyser pour regrouper des entités ou des concepts.

Java EE

- Correspond une spécification technique Java d'Oracle
- est construite sur le langage Java et la plate-forme Java SE
- Fournit un ensemble d’API (Servlet, JSP, JMS, JPA, EJB, batch…) - des outils
- Serveurs d’application (Jboss, Websphere, Weblogic) et un serveur de servlet (Tomcat).
- Exemple d’utilisation : le site est dit « dynamique ».
- Avantages :
  - Gestion des pools de connexions JDBC
  - Gestion des API

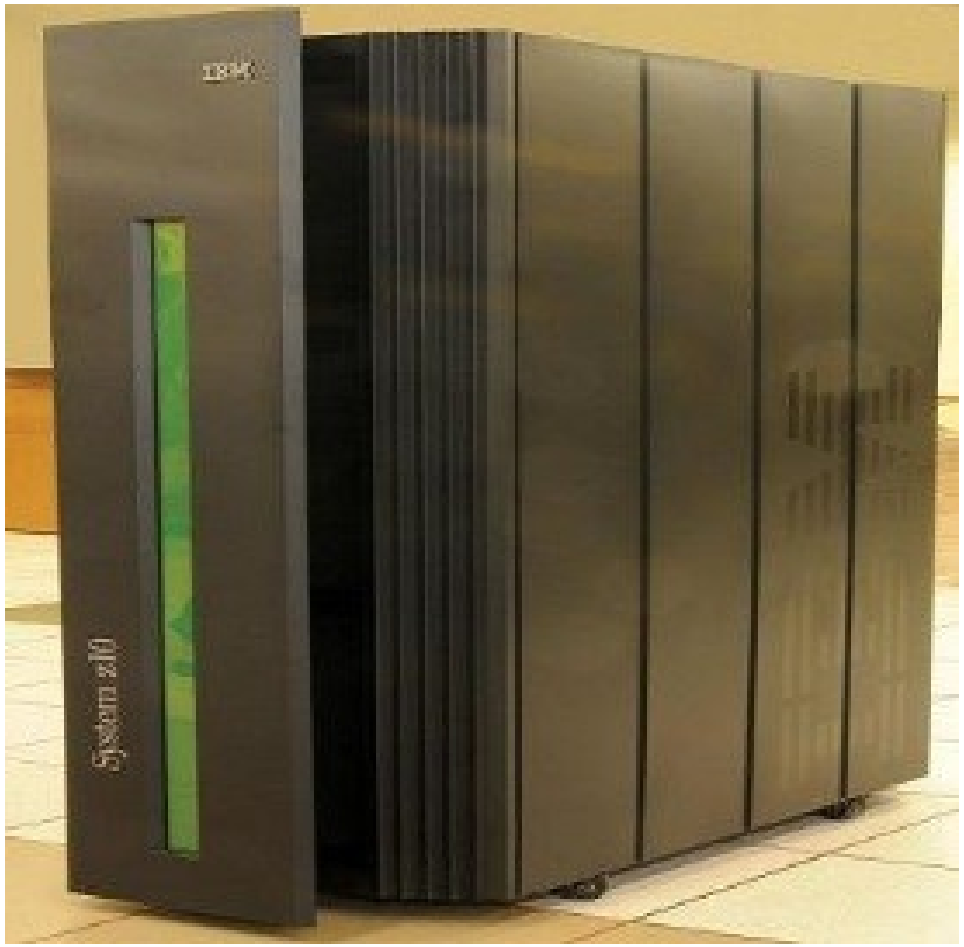
Java EE

- Assez complexe.
- D'autres alternatives : Spring. Un conteneur léger.



SOA (Service Oriented Architecture)

- Evolution des systèmes d'information
  - Mainframe (Année 60-70)
    - Assure une haute disponibilité
    - Mais peu évolutif et coût élévé.
  - Client lourd / serveur (Année 80)
    - Coût moindre
    - Complexité dans le déploiement
    - Complexité si le SI n'est pas correctement défini.



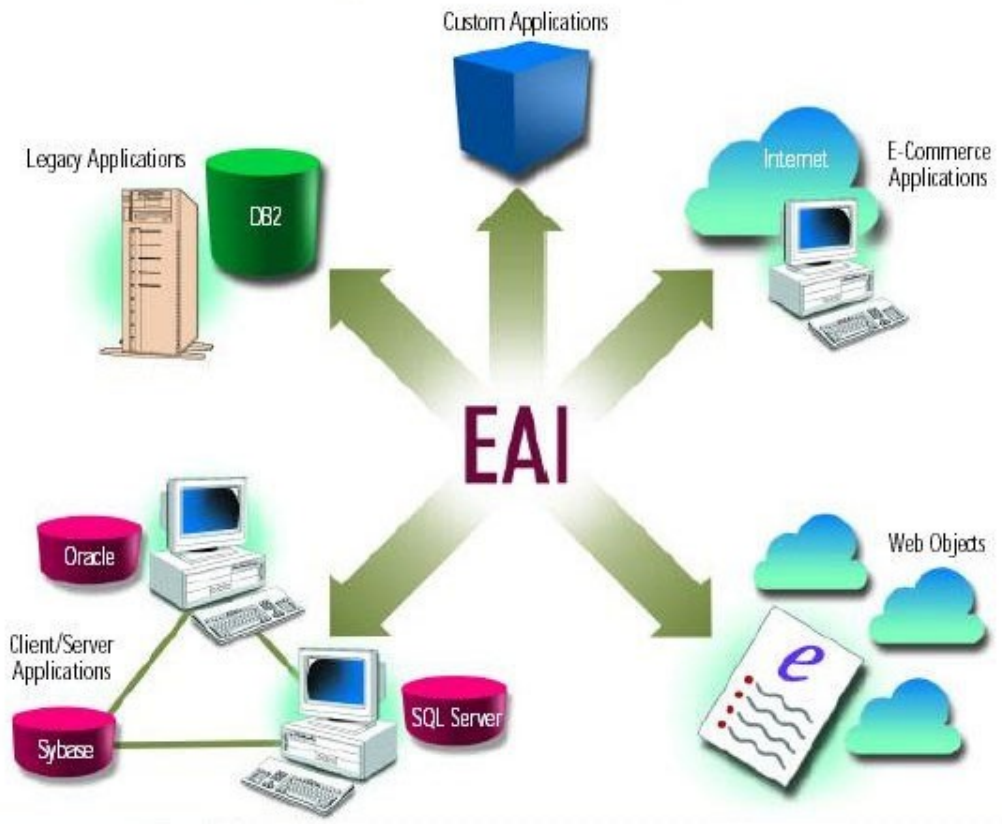
SOA (Service Oriented Architecture)

- Evolution des systèmes d’information
  - Client Web/ serveur
    - Retour en arrière. Traitement coté serveur.
    - Simplicité et réduit les coûts
    - Maintenances
  - Le cloud
    - Sous traitance d’une partie du SI
    - Inconvénient : sécurité, intégration avec le SI interne, droit des données.

SOA (Service Oriented Architecture)

- Début des années 2000, forte augmentation des applications hétérogènes.
  - Mise en place des interlogiciels « middlewares » : permet la communication entre les applications informatiques.
  - Exemple : communication entre les logiciels

- EAI (Enterprise Application Integration) : Favorise la transmission des messages entre les applications
  - Hub / gros connecteur propriétaire
  - Inconvénients : complexe à créer et à maintenir, la norme mal définie, coût budgétaire élevé.

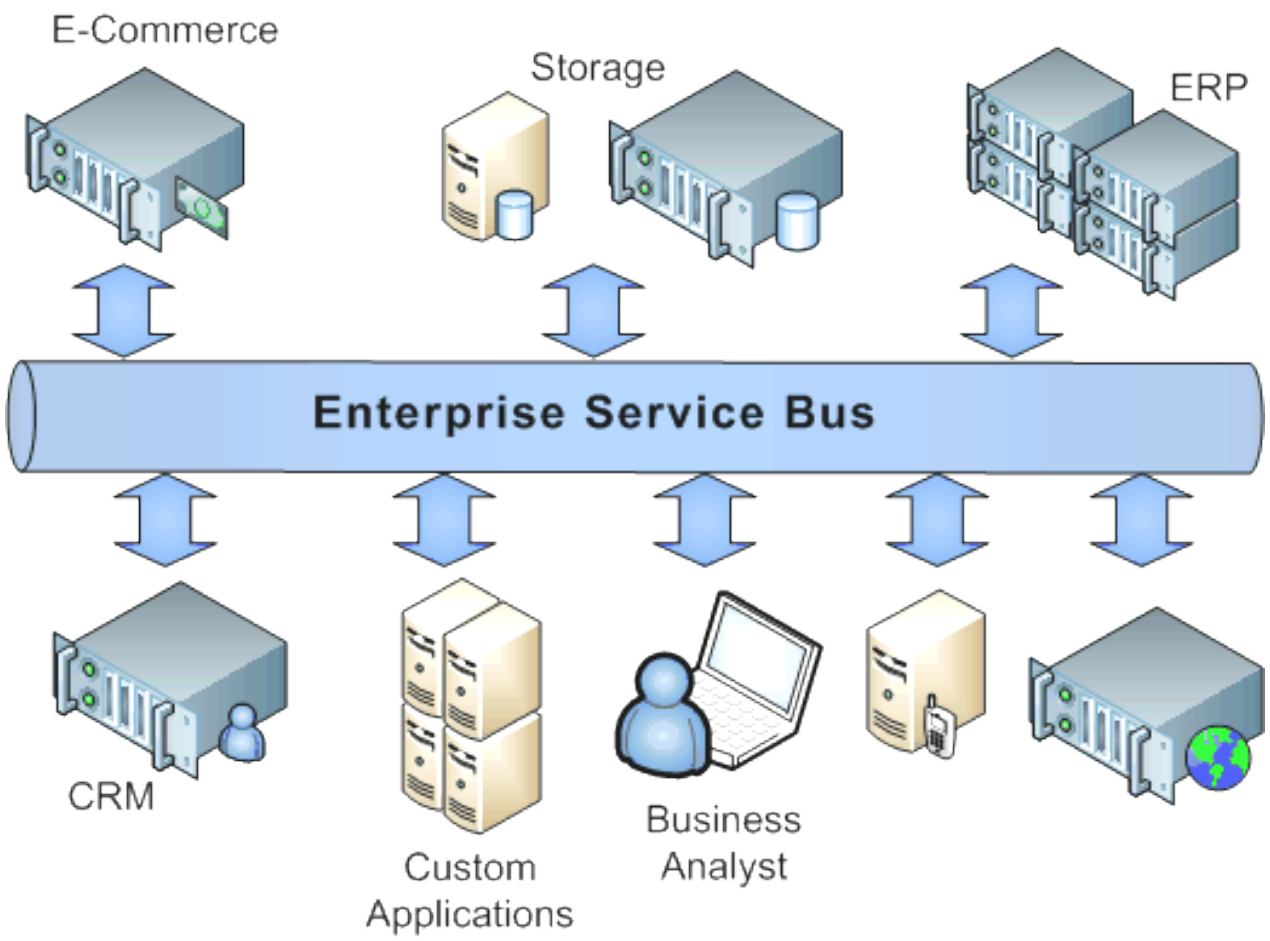


SOA (Service Oriented Architecture)

ESB : Evolution EAI

- Bus d'application : permettant de relier toutes les applications d'une entreprise autour d'un canal commun

- Principes :
  - Pouvoir se connecter vers des protocoles ou de format de données hétérogènes
  - Routage : transférer les messages (consommateur / Producteur).
  - Pouvoir transformer un format à un autre.
  - Pouvoir monitorer les messages qui transitent.



SOA (Service Oriented Architecture)

- SOA
  - Philosophie : concept
  - Basé sur des services
  - Service :
    - Sans état
    - Couplage faible (n’appelle pas un autre service) : autonomie
    - Forte Valeur Ajoutée
    - Web SOAP, Rest, … ou pas.
    - Réutilisable
    - Contrat
    - invocable à distance par n’importe quel programme
    - Annuaire : localisation

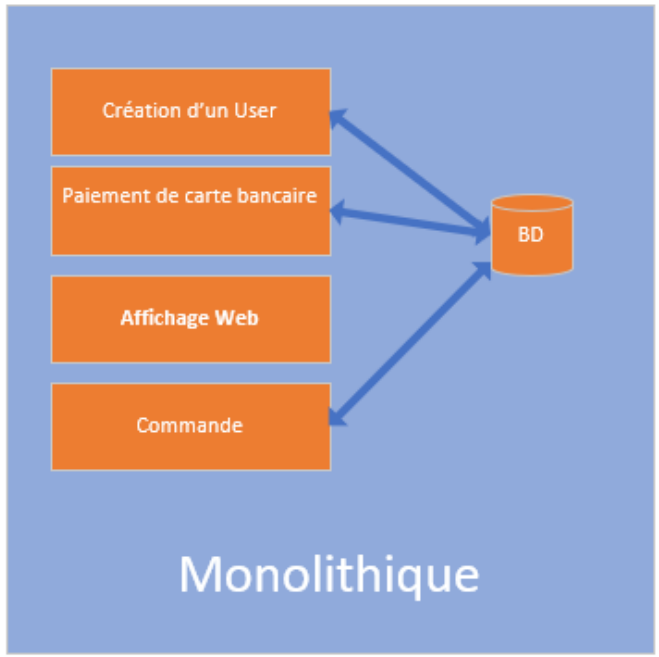
  - ESB a adopté cette philosophie.

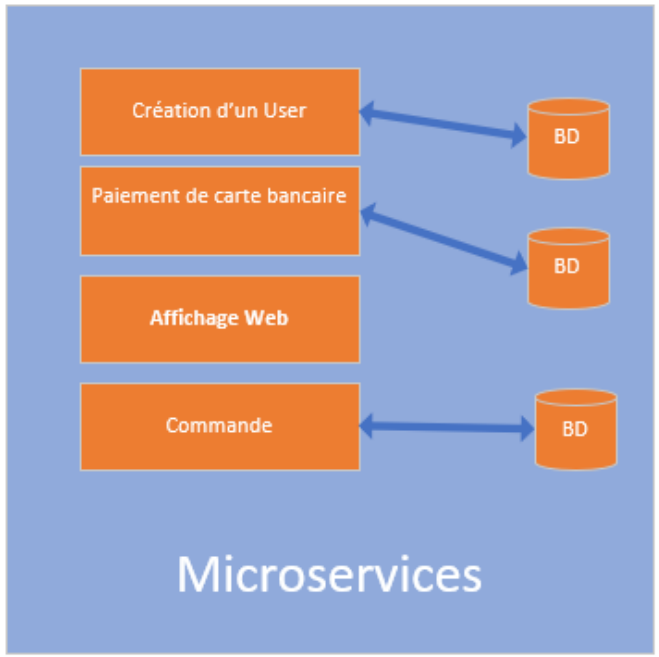
Microservices

⇒**Evolution du SOA**

Le + :

- **Scalable**
- **Facilité des mises à jour**

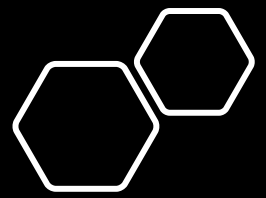




Microservices

Les avantages :

- découper une application en petits services autonomes
- privilégier la construction d'applications autonomes
  - Une seule base de donnée
  - Un serveur autonome
- Déploiement plus simple
- Simplification des évolutions
- Meilleur gestion des pannes



Microservices

Les inconvénients :

- La mise en œuvre des tests
- Peut devenir complexe sur le volume des microservices
- Latences du réseau, …
- La gestion des transactions distribuées
- La mise en place de ce système : bonne connaissance des développeurs

Docker

C'est quoi

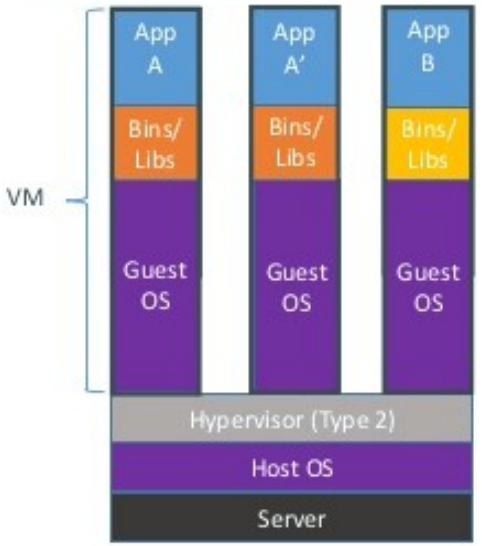
- est une plate-forme logicielle
- de déployer des applications rapidement
- De dimensionner des applications dans n'importe quel environnement

Concept

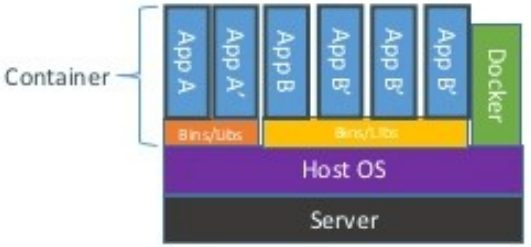
- de créer des environnements (appelées containers) de manière à isoler des applications.

Docker

Containers vs. VMs



Containers are isolated, but share OS and, where appropriate, bins/libraries



docker